

Compatibilidad entre consumidor y proveedor

Servicio de reservaciones e inventario con PactV3 y Vitest

Entrega académica

CONSUMER reservation-consumer	CONTRATO PactV3 / pacts/*.json	PROVIDER inventory-provider
-------------------------------	--------------------------------	-----------------------------

El consumidor consulta GET /inventory/:sku antes de aceptar una reservación. El contrato se genera contra un mock de Pact y luego se verifica contra el servidor HTTP real del proveedor.

Resultado de validación

Consumidor: 3 interacciones aprobadas. Proveedor: 3 interacciones aprobadas contra la implementación real, incluyendo los estados de disponibilidad, sin existencias y SKU inexistente.

1. Frontera y arquitectura

La frontera elegida es una API HTTP entre dos aplicaciones independientes. El consumidor no conoce el almacén del proveedor: solo interpreta el código HTTP y el JSON de GET /inventory/:sku. El proveedor expone un servidor Node HTTP real con un almacén determinista en memoria.

Consumidor	Contrato	Proveedor
InventoryClient fetch real canAcceptReservation	PactV3 MatchersV3 3 interacciones	GET /inventory/:sku stateHandlers Verifier

Estados del dominio

Estado	SKU	Respuesta	Resultado
inventory has stock for SKU	SKU-AVAILABLE	200, quantity 7	Acepta
inventory has no stock for SKU	SKU-OUT	200, quantity 0	Rechaza
SKU does not exist in inventory	SKU-MISSING	404, error + sku	Nulo

2. Generación del contrato

El cliente HTTP real se ejecuta contra el servidor simulado de Pact. MatchersV3 permite validar el tipo y la forma de los datos variables: string para identificadores y nombres, integer para cantidades y boolean para disponibilidad.

```
provider.addInteraction({
  states: [{ description: "inventory has stock for SKU" }],
  uponReceiving: "a request for available inventory",
  withRequest: { method: "GET", path: "/inventory/SKU-AVAILABLE" },
  willRespondWith: {
    status: 200,
    body: { sku: string("SKU-AVAILABLE"),
      name: string("Conference room A"),
      quantity: integer(7), available: boolean(true) }
  }
});

await provider.executeTest(async (mockServer) => {
  const client = new InventoryClient(mockServer.url);
  await expect(client.canAcceptReservation("SKU-AVAILABLE", 2))
    .resolves.toBe(true);
});
```

Extracto de consumer/tests/inventory.pact.test.ts

Resultado observado: `inventory.pact.test.ts (3 tests)`, los tres requests coinciden con el mock y se genera `pacts/reservation-consumer-inventory-provider.json`.

```
"consumer": { "name": "reservation-consumer" },
"provider": { "name": "inventory-provider" },
"interactions": [
  { "description": "a request for an unknown SKU",
    "providerStates": [{ "name": "SKU does not exist in inventory" }],
    "request": { "method": "GET", "path": "/inventory/SKU-MISSING" },
    "response": { "status": 404 } },
  ... 2 interacciones adicionales
]
```

Extracto legible del Pact generado

3. Verificación del proveedor real

La prueba arranca el servidor del proveedor en un puerto local, registra stateHandlers para cada estado del dominio y ejecuta Verifier con el Pact generado. Por tanto, las respuestas pasan por la implementación real de GET /inventory/:sku.

```
Verifying a pact between reservation-consumer and inventory-provider

Given SKU does not exist in inventory
  status code 404 (OK)
  matching body (OK)

Given inventory has stock for SKU
  status code 200 (OK)
  matching body (OK)

Given inventory has no stock for SKU
  status code 200 (OK)
  matching body (OK)

provider.verification.test.ts (1 test)
1 passed
```

Extracto de la ejecución de npm run test:provider

La verificación confirma códigos HTTP, encabezados y cuerpos en los tres escenarios. El workflow de GitHub Actions repite typecheck, pruebas del consumidor y verificación del proveedor en cada push y pull request.

Enlaces de entrega

Repositorio público	https://github.com/ZaqueoChivalan/-S9-Contrato-consumidor-proveedor-con-Pact
GitHub Actions	https://github.com/ZaqueoChivalan/-S9-Contrato-consumidor-proveedor-con-Pact/actions/runs/35482450795
Video <= 3 minutos	https://drive.google.com/file/d/15PqFFNDDEeJYe82dScSbv6Sf5-XcHKnD/view?usp=sharing

La entrega incluye el repositorio público, la ejecución exitosa de CI y el video demostrativo.